Refs\Running_SlashCommand_MP4+WizWhisp_in_Parallel.md

perplexity

Table of Contents

- I have a repo that creates transcripts of MP4s… (Perplexity answer)
  - Strategy: keep one repo, fan out jobs
  - Things to watch for
  - Example minimal pattern (no extra clones)
- Running MP4+WizWhisp on 2 MP4s in Parallel Without Mixing Outputs
  - The two failure modes
  - What the slug already protects (verified)
  - The two gaps the slug does NOT cover
  - How to actually run two in parallel
    - Option A — two separate Claude Code instances (recommended, cleanest)
    - Option B — one orchestrator agent, two subagents
    - Option C — sequential (zero risk, if the machine is tight)
  - Resource caveats (why “more parallel” isn’t always faster)
  - Pre-flight checklist to guarantee no mixing

I have a repo that creates transcripts of MP4s. I want to process multiple MP4s in parallel. Is there a way (other than cloning the repo to multiple directories) by which I can do that?

Yes — you can absolutely parallelize this without cloning the repo; the right approach is to treat your transcript generator as a function and run multiple instances via a job runner (Python multiprocessing, GNU parallel, etc.) that point at the same codebase.[^1]

I’ll assume the repo is either:

- a CLI that takes `input.mp4` and writes `output.json`/`output.txt`, or
- a Python script/module you’re invoking directly.

Strategy: keep one repo, fan out jobs

Core idea: keep a single working tree and run multiple *processes* or *threads* against different input files, ensuring each run writes to a unique output path.[^1]

You have three main options:

- **Python-level parallelism (if the repo is Python)** Wrap the repo’s main transcript function/CLI call in something like `concurrent.futures.ProcessPoolExecutor` or `multiprocessing`:

```
from concurrent.futures import ProcessPoolExecutor
import subprocess
from pathlib import Path

def transcribe_one(mp4_path: Path, out_dir: Path):
    out = out_dir / (mp4_path.stem + ".json")
    subprocess.run(
        ["python", "transcribe.py", str(mp4_path), str(out)],
        check=True,
    )

mp4s = list(Path("input_videos").glob("*.mp4"))
out_dir = Path("transcripts")
out_dir.mkdir(exist_ok=True)

with ProcessPoolExecutor(max_workers=4) as ex:
    ex.map(lambda p: transcribe_one(p, out_dir), mp4s)
```

All workers share the same repo; isolation is via process boundaries and distinct input/output files.

- **Shell-level parallelism (CLI-style repo)** If the repo exposes a CLI like `transcribe input.mp4 -o output.json`, you can use GNU `parallel` or `xargs -P` from one repo checkout.[^2]

Example with `parallel`:

```
cd /path/to/repo

ls input_videos/*.mp4 | parallel -j4 \
  'python transcribe.py {} transcripts/{/.}.json'
```

Or with `xargs`:

```
ls input_videos/*.mp4 | xargs -P4 -I{} \
  python transcribe.py {} transcripts/$(basename {} .mp4).json
```

`-P4`/`-j4` controls concurrency; increase until you saturate CPU/GPU/IO.[^4]

- **Job queue / orchestrator layer** Since you're already comfortable with tmux and multi-agent workflows, you could:
  - run multiple tmux panes each executing a subset of files, or
  - have a simple queue manager script that pops file paths and spawns worker processes until N concurrent jobs are running.

This is effectively recreating a primitive batch system like `GNU parallel` in Python or bash but keeps all logic near the repo.

Things to watch for

- **Shared state**: Make sure the transcript code doesn't write to a single fixed file, DB, or temp path without unique naming. If it does, parameterize those paths so each process writes to its own location.
- **GPU vs CPU**: If transcription uses a GPU model (e.g., Whisper), you might get better throughput by:
  - one process with internal batching, or
  - a small number of workers (e.g., 1–2) rather than per-core processes, depending on VRAM and model size.[^5]
- **I/O concurrency**: If MP4s are large and on spinning disks or network storage, too many workers will thrash the disk; scale `-j`/`max_workers` accordingly.[^4]

Example minimal pattern (no extra clones)

If you just want a quick win and your repo exposes `transcribe.py`:

```
cd /path/to/transcript-repo

mkdir -p transcripts

find ./mp4s -maxdepth 1 -name '*.mp4' \
  | parallel -j$(nproc) \
      'python transcribe.py {} transcripts/{/.}.json'
```

Single repo, many concurrent jobs, each reading a different MP4 and writing a different output.[^2]

What's the current entry point you're using (Python function, CLI script, or something else)? That will let me sketch a concrete parallelization wrapper that drops straight into your setup.

⁂

Running MP4+WizWhisp on 2 MP4s in Parallel Without Mixing Outputs

> The Perplexity answer above addresses a *different* question than the one that matters here — it assumes the repo is a `transcribe.py` CLI you can fan out with `parallel`/`xargs`. **MP4+WizWhisp is not that.** It is an **agent-executed slash command**: the "program" running it is a Claude Code instance that reads frames with its eyes, curates by judgment, and writes files across ~8 phases. So the parallelism question has two independent axes, and you must isolate **both** or the outputs mix:

1. **Filesystem isolation** — do the two runs write to disjoint paths?
2. **Context isolation** — is each run driven by its own agent, or is one agent juggling both videos in a single attention span?

After checking the command and all four helper scripts: the filesystem axis is already almost fully isolated by design, and there are no hidden shared temp paths.

The two failure modes

Failure mode 1 — path collision (filesystem). Every intermediate and output path in MP4+WizWhisp is namespaced by the **slug** (the MP4 filename without extension, spaces→underscores). If two runs share a slug, they stomp each other's files.

Failure mode 2 — attention collision (context). If you invoke `/MP4+WizWhisp fileA` and `/MP4+WizWhisp fileB` in the **same** Claude session, one agent interleaves both jobs. Even with perfect path separation, it can attach fileA's caption to fileB's frame, or build one `picks.json` from a mental blend of both. This is the more insidious one, and no amount of path hygiene fixes it — it is fixed only by giving each video its own agent/process.

What the slug already protects (verified)

Every per-run artifact carries the slug, so two videos with **different filenames** never touch the same file:

Artifact	Path	Namespaced?
Raw frames	`Input/{slug}_figs/`	✅ slug
Contact sheets	`Input/_montage_{slug}/`	✅ slug
Picks list	`Input/{slug}_picks.json`	✅ slug
Curated figures	`Input/{slug}_figs_filtered/`	✅ slug
Frame manifest	`Input/{slug}_frames_manifest.json`	✅ slug
Raw transcript (DB fallback)	`Input/{slug}_RAW_TRANSCRIPT.txt`	✅ slug
Deck source (temp)	`Output/Slides/_{slug}_deck_src.html`	✅ slug

The helper scripts hide no fixed scratch paths — `mp4wizwhisp_montage.py`, `mp4wizwhisp_curate.py`, `inline_reveal.py`, and `embed_slide_images.py` all take their input/output/manifest/picks paths as explicit `sys.argv` arguments (all slug-derived). The only shared resource any of them touches is **`.cache/reveal-5.1.0/`**, which `inline_reveal.py` opens **read-only** — two concurrent runs reading it is fine. The WizWhisp SQLite DB (`TaskHistory.db`) is likewise read-only. So there is no shared-mutable-state landmine at the script level.

Net: as long as the two slugs differ, Phases 0–4 and Phase 7's temp file are collision-proof.

The two gaps the slug does NOT cover

These are the only real ways to mangle outputs on the filesystem, and you must guard both:

Gap 1 — identical MP4 basenames → identical slugs. If both files are named `recording.mp4` (in different folders), they produce the **same slug** and every row in the table above collides. Fix before starting: rename one input so the basenames differ (e.g. `recording_A.mp4` / `recording_B.mp4`), or copy them into `Input/` under distinct names.

Gap 2 — the *final* filenames come from the H1 title, not the slug. Phase 5 derives the output name from the content-derived H1:

- `Output/Formatted/{Title_Case}_FORMATTED_TRANSCRIPT.md`
- `…_FORMATTED_TRANSCRIPT.html`
- `Output/Slides/{Title}_{N}_SLIDES.html`

Two topically similar videos could yield near-identical titles → the second run overwrites the first's `.md`/`.html`. The slug protects the *figures*, but not these. Guard it by passing an explicit `--title "..."` to each run (or, if driving via subagents, tell each to prefix/suffix the title distinctly), so the two title-derived filenames cannot converge.

How to actually run two in parallel

In rough order of robustness:

Option A — two separate Claude Code instances (recommended, cleanest)

Open two terminals (or two tmux panes) in this same repo. In each, run one invocation:

```
# terminal 1
/MP4+WizWhisp "Input/videoA.mp4" "Input/videoA_timestamped.txt" --title "Topic A ..."
```

```
# terminal 2
/MP4+WizWhisp "Input/videoB.mp4" "Input/videoB_timestamped.txt" --title "Topic B ..."
```

Each instance is its own OS process with its own context window → **full context isolation** (fixes failure mode 2), and distinct slugs + distinct `--title` (fixes both gaps). No repo cloning needed — exactly the "keep one repo, fan out processes" idea from the Perplexity note, but the "process" is a whole Claude instance, not a `transcribe.py`. This is the approach to prefer.

Option B — one orchestrator agent, two subagents

Spawn two subagents in a single message (so they run **concurrently**), each handed: the full MP4+WizWhisp procedure, one video + its transcript, its own slug, and a distinct title. Each subagent has an isolated context, so no attention-mixing; their writes are slug-disjoint. Collect both result summaries at the end. This keeps everything inside one session and requires no second terminal — the tradeoff is that subagents cannot literally invoke the `/MP4+WizWhisp` *slash command*, so you pass them the procedure text to execute.

Option C — sequential (zero risk, if the machine is tight)

Just run one, then the other. Slower, but no CPU/IO contention and no chance of mixing.

Resource caveats (why "more parallel" isn't always faster)

The long pole is **Phase 2 frame extraction** — native-resolution JPEG dumps via ffmpeg, which is CPU- and disk-heavy. Two concurrent extractions on one machine are fine; pushing to 4–8 will thrash the disk and slow *all* of them (the Perplexity note's `-j$(nproc)` advice assumes cheap CPU-bound jobs, which frame extraction on a laptop SSD is not). pandoc and the Python inliners are light by comparison. So: **2 in parallel = good; treat it as the ceiling on a single workstation** unless you watch disk throughput.

Pre-flight checklist to guarantee no mixing

1. **Distinct slugs** — confirm the two MP4 basenames differ (Gap 1).
2. **Distinct `--title`** on each invocation — so the title-derived output filenames cannot collide (Gap 2).
3. **Provide the transcript path explicitly** to each run when you have it, so neither run has to guess-match the WizWhisp DB by basename (removes any ambiguity in the DB-fallback path).
4. **Separate agents/processes** — Option A or B, never two `/MP4+WizWhisp` calls in one session.
5. On finish, the two decks/transcripts land in the shared `Output/Formatted` and `Output/Slides` dirs but under distinct names, and the two `Input/{slug}_figs_filtered/` dirs stay separate — verify both `.md` files exist with different names before cleanup.